Amazon Web Services and Compute

Amazon Web Services (AWS) is a global cloud computing platform offering on-demand services such as compute, storage, networking, databases, analytics, machine learning, and security. This chapter focuses on **AWS Compute services**, starting with **Amazon EC2**, and expanding to load balancing, auto scaling, containers, and serverless computing.

Topics Covered

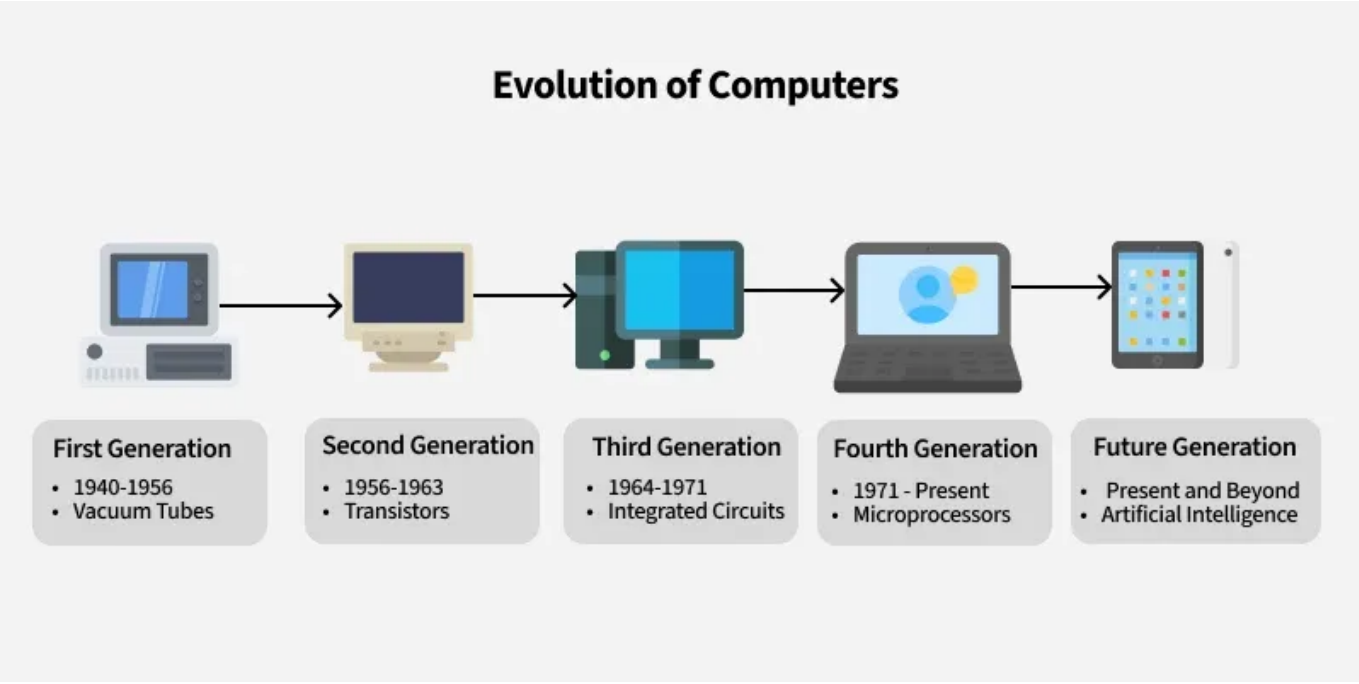
- Evolution of computing from physical machines to cloud
- AWS Global Cloud Infrastructure
- EC2 instance provisioning (Console and CloudShell)
- Elastic Load Balancers (ELB) and Auto Scaling Groups (ASG)
- AWS compute evolution: EC2 → Containers → Serverless

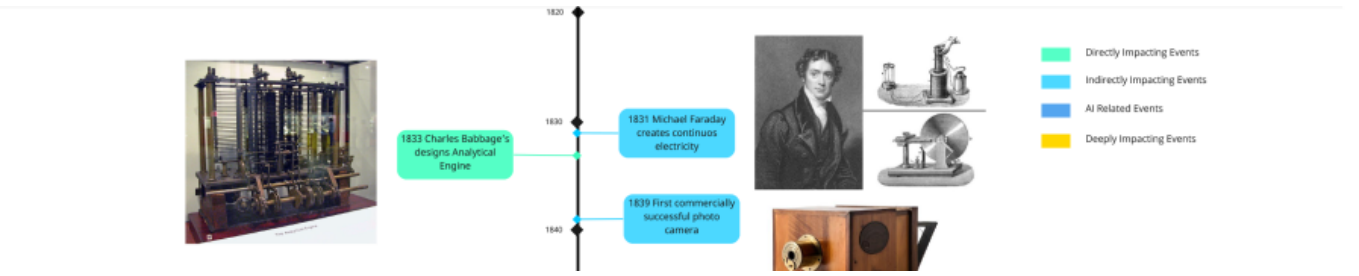
History of Computing

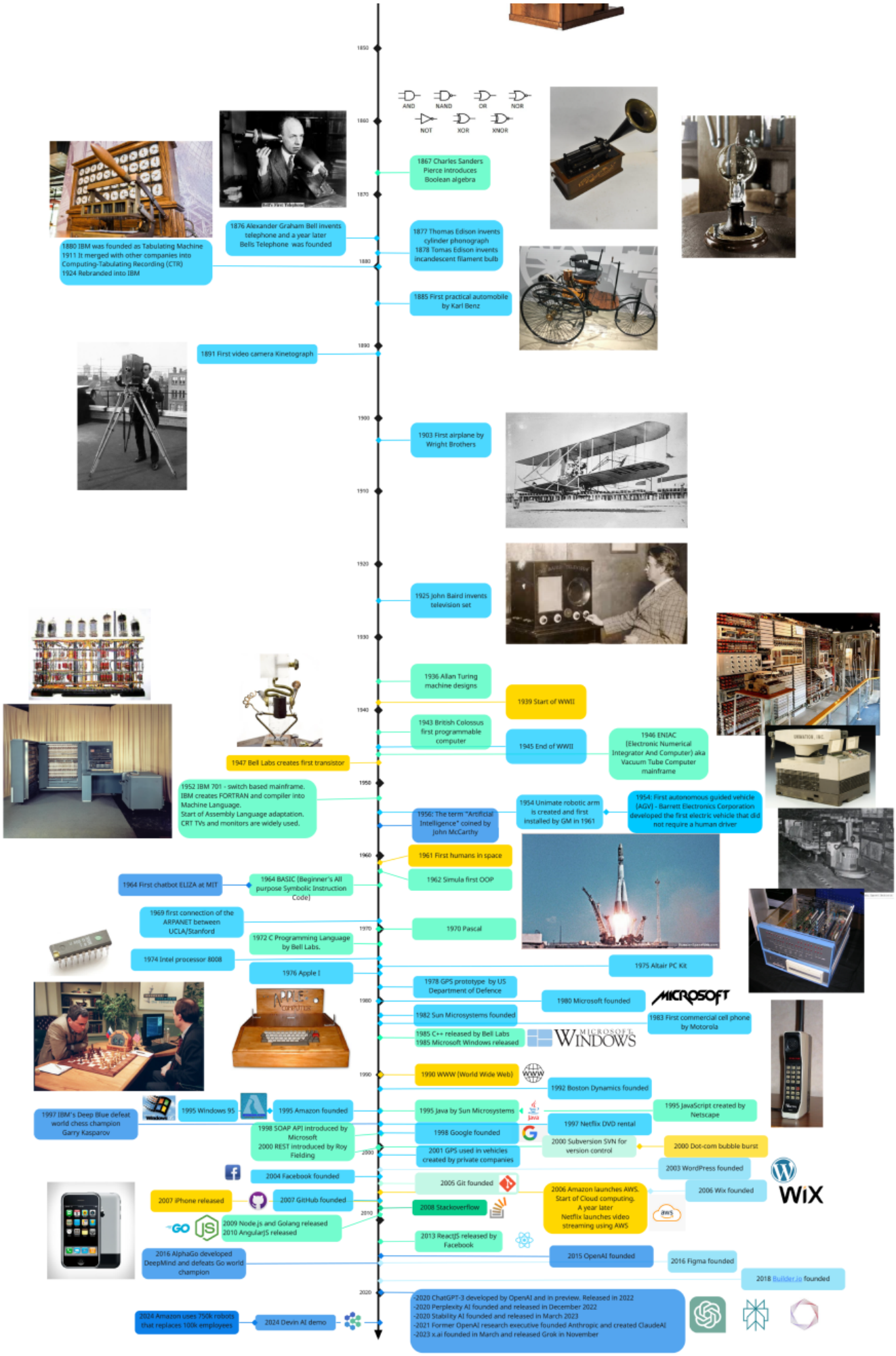
The First Computers

The first practical computer, **ENIAC (1945)**, marked the beginning of digital computing. Over 75+ years, computers evolved from massive physical machines into portable, virtual, and disposable resources.

Figure 1.1 – Computer evolution milestones






1850
1860
1870
1880
1890
1900
1910
1920
1930
1940
1950
1960
1970
1980
1990
2000
2010
2020
AND
NAND
OR
NOR
NOT
XOR
XNOR
1867 Charles Sanders Pierce introduces Boolean algebra
1876 Alexander Graham Bell invents telephone and a year later Bells Telephone was founded
1877 Thomas Edison invents cylinder phonograph
1878 Tomas Edison invents incandescent filament bulb
1880 IBM was founded as Tabulating Machine
1911 It merged with other companies into Computing-Tabulating Recording (CTR)
1924 Rebranded into IBM
1885 First practical automobile by Karl Benz
1891 First video camera Kinetograph
1903 First airplane by Wright Brothers
1925 John Baird invents television set
1936 Allan Turing machine designs
1939 Start of WWII
1943 British Colossus first programmable computer
1945 End of WWII
1946 ENIAC (Electronic Numerical Integrator And Computer) aka Vacuum Tube Computer mainframe
1947 Bell Labs creates first transistor
1952 IBM 701 - switch based mainframe.
IBM creates FORTRAN and compiler into Machine Language.
Start of Assembly Language adaptation.
CRT TVs and monitors are widely used.
1954 Unimate robotic arm is created and first installed by GM in 1961
1954: First autonomous guided vehicle (AGV) - Barrett Electronics Corporation developed the first electric vehicle that did not require a human driver
1956: The term "Artificial Intelligence" coined by John McCarthy
1961 First humans in space
1962 Simula first OOP
1964 First chatbot ELIZA at MIT
1964 BASIC (Beginner's All purpose Symbolic Instruction Code)
1969 first connection of the ARPANET between UCLA/Stanford
1970 Pascal
1972 C Programming Language by Bell Labs.
1974 Intel processor 8008
1975 Altair PC Kit
1976 Apple I
1978 GPS prototype by US Department of Defence
1980 Microsoft founded
1982 Sun Microsystems founded
1983 First commercial cell phone by Motorola
1985 C++ released by Bell Labs
1985 Microsoft Windows released
1990 WWW (World Wide Web)
1992 Boston Dynamics founded
1995 Windows 95
1995 Amazon founded
1995 Java by Sun Microsystems
1995 JavaScript created by Netscape
1997 IBM's Deep Blue defeat world chess champion Garry Kasparov
1997 Netflix DVD rental
1998 SOAP API introduced by Microsoft
2000 REST introduced by Roy Fielding
1998 Google founded
2000 Subversion SVN for version control
2000 Dot-com bubble burst
2001 GPS used in vehicles created by private companies
2003 WordPress founded
2004 Facebook founded
2005 Git founded
2006 Amazon launches AWS. Start of Cloud computing.
A year later
Netflix launches video streaming using AWS
2006 Wix founded
2007 iPhone released
2007 GitHub founded
2008 Stackoverflow
2009 Node.js and Golang released
2010 AngularJS released
2013 ReactJS released by Facebook
2015 OpenAI founded
2016 Figma founded
2016 AlphaGo developed DeepMind and defeats Go world champion
2018 Builder.io founded
2020 ChatGPT-3 developed by OpenAI and in preview. Released in 2022
2020 Perplexity AI founded and released in December 2022
2020 Stability AI founded and released in March 2023
2021 Former OpenAI research executive founded Anthropic and created ClaudeAI
2023 x.ai founded in March and released Grok in November
2024 Amazon uses 750k robots that replaces 100k employees
2024 Devin AI demo

Computer Architecture Basics

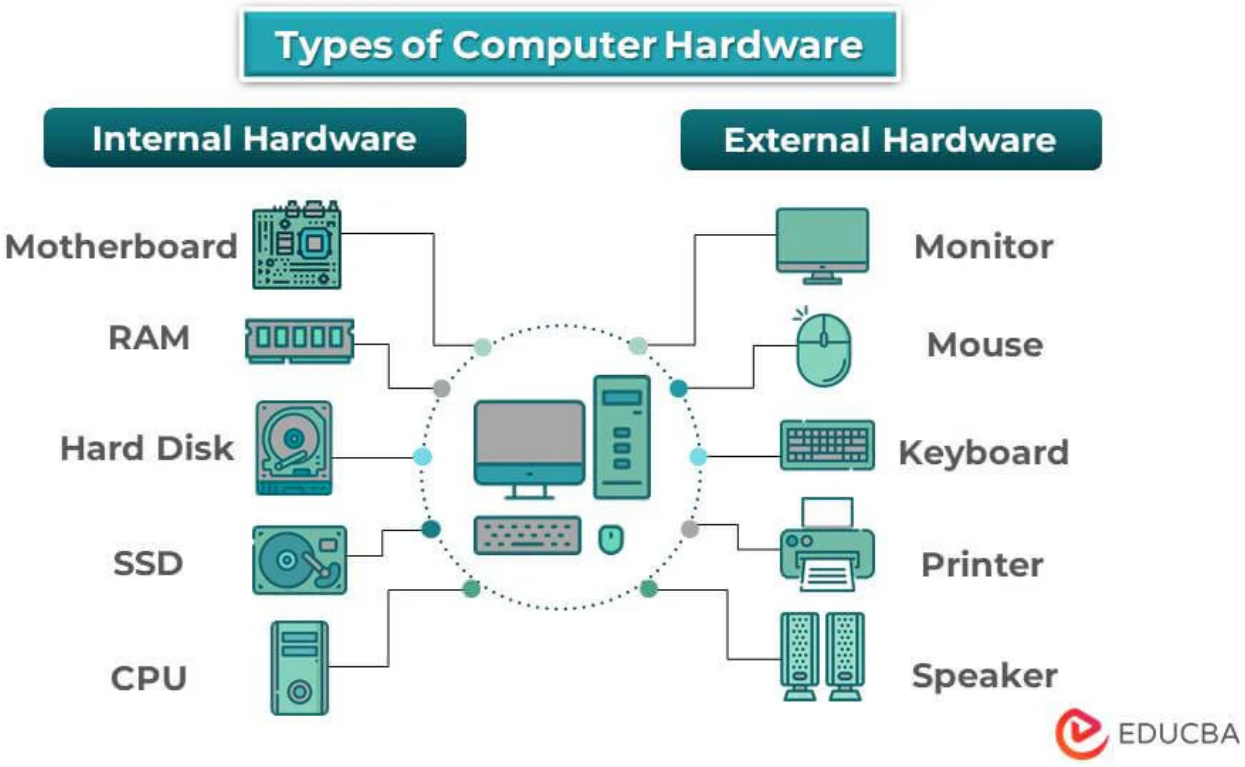
A traditional computer consists of:

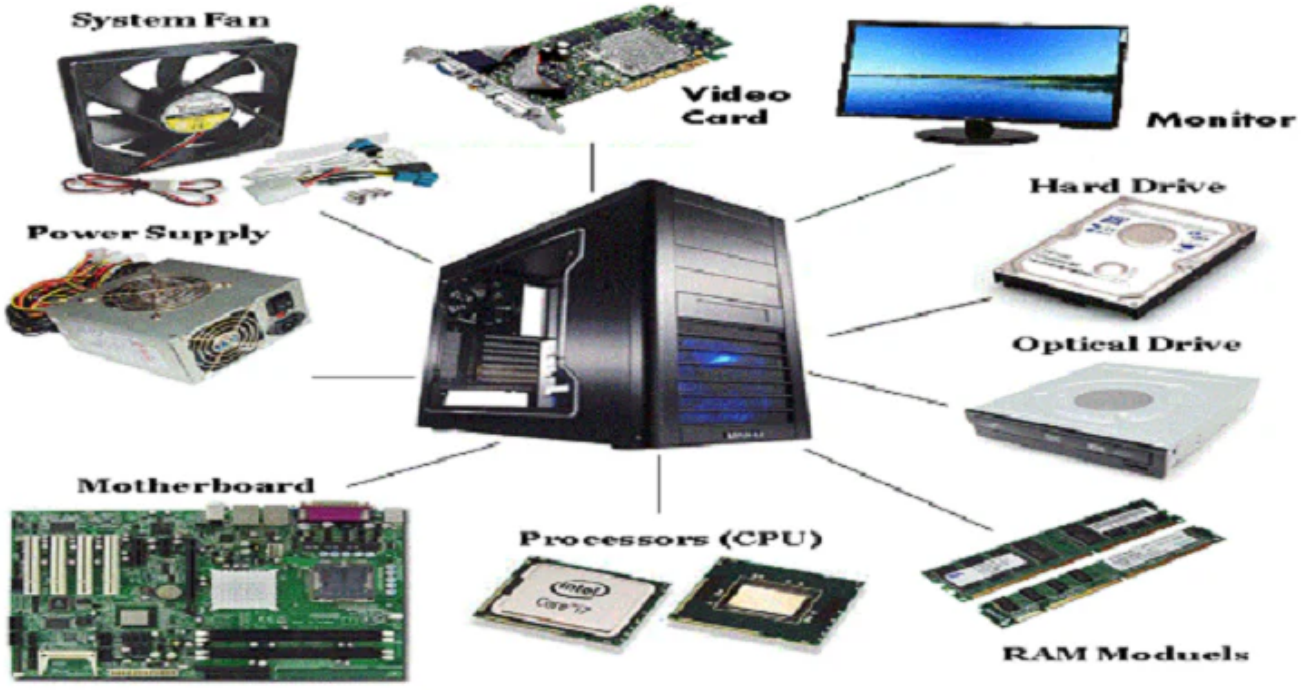
- CPU
- RAM
- Hard Disk
- Network Interface Card (NIC)

These components operate together with:

- Operating systems (Windows, Linux, macOS)
- Application software (databases, servers, productivity tools)

Figure 1.2 – Internal computer hardware components





Data Centers

Data centers aggregate compute, storage, and networking resources to serve organizational IT needs.
Limitations of traditional data centers:

- High capital and maintenance costs
- Long provisioning cycles
- Limited scalability

Large-scale projects (for example, genome analysis) are infeasible without cloud-scale resources.

Virtual Machines (VMs)

Virtualization, popularized by VMware (1998), enabled:

- Multiple VMs on one physical server
- Hardware abstraction via hypervisors
- VM portability and live migration (vMotion)

This abstraction layer made **cloud computing possible**.

Cloud Computing Concept

Cloud computing is based on:

- **On-demand provisioning**
- **Pay-as-you-go pricing**
- **Elastic scalability**

Pets vs Cattle Model

- **Traditional IT**: servers treated as *pets* (manually repaired)
- **Cloud**: servers treated as *cattle* (automatically replaced)

Cloud infrastructure is:

- Global
- Elastic and scalable
- Highly available
- Secure
- Cost-efficient

Infrastructure is defined programmatically using **Infrastructure as Code (IaC)**.

Amazon EC2 Overview

EC2 (launched in 2006) allows users to rent virtual machines with customizable:

- Instance types (CPU, RAM, network)
- Operating systems
- Storage and networking options

Key milestones:

- **2013**: Reserved Instances (cost optimization)
- **2017**: EC2 Fleet (multi-AZ, multi-instance orchestration)

Computer Evolution Path

Computing evolved from:

- Physical machines → Virtual machines → Disposable cloud resources
- Physical data centers → Software-defined, globally deployable infrastructure

---

Amazon Global Cloud Infrastructure

AWS operates global infrastructure composed of:

- **Regions**: geographic locations
- **Availability Zones (AZs)**: isolated data center groups within regions

Characteristics:

- Fault isolation
- High-speed private networking
- Regional and AZ-level deployment control

Example AZ naming: `us-east-1d`

Building EC2 Instances

AWS Account Setup (Prerequisites)

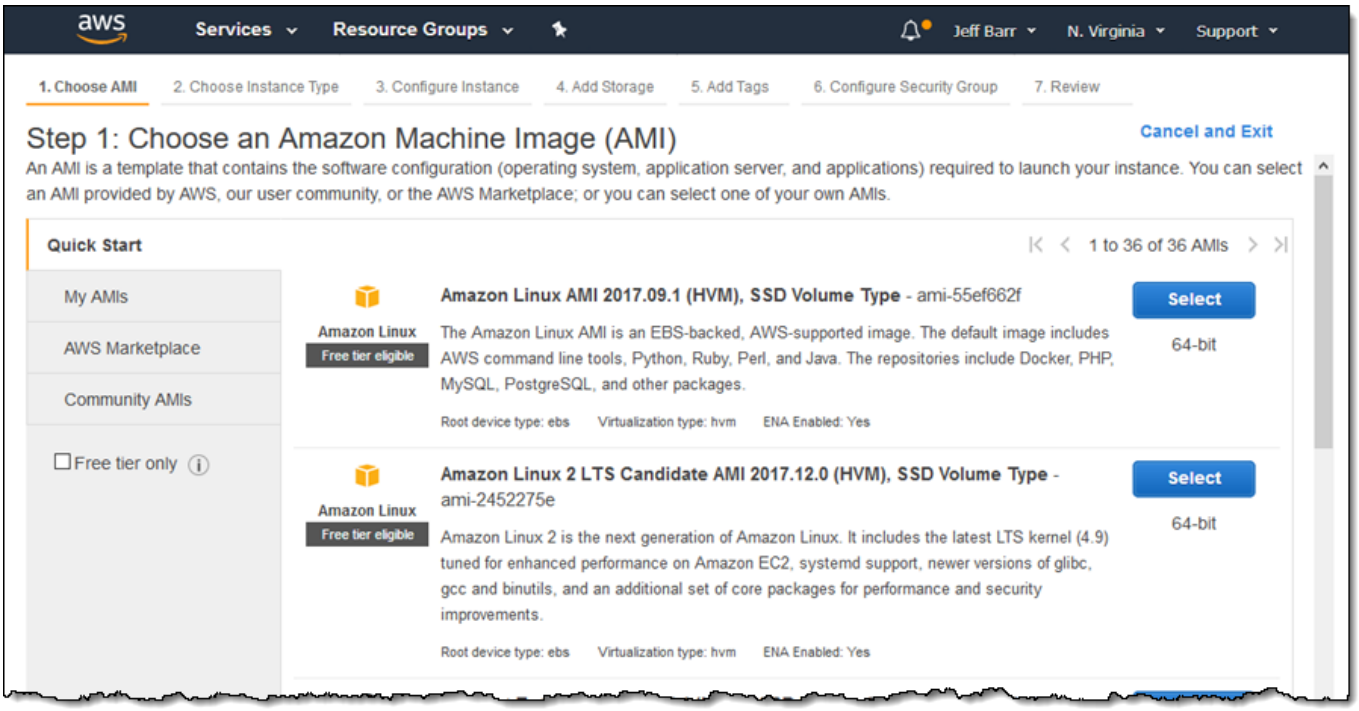
- Enable MFA [Multi Factor Authentication]
- Avoid routine use of root account
- Delete unused resources to prevent charges

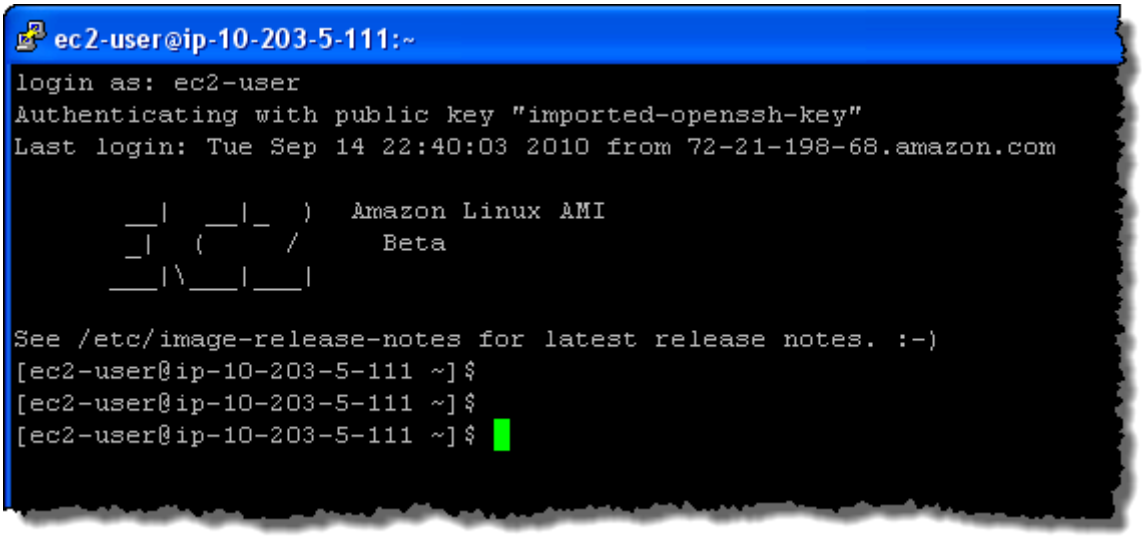
Launching EC2 via AWS Console

Step-by-Step Summary

1. **Select AMI (Software Image)** AMIs define OS and preinstalled software.

Figure 1.3 – Selecting Amazon Linux 2 AMI

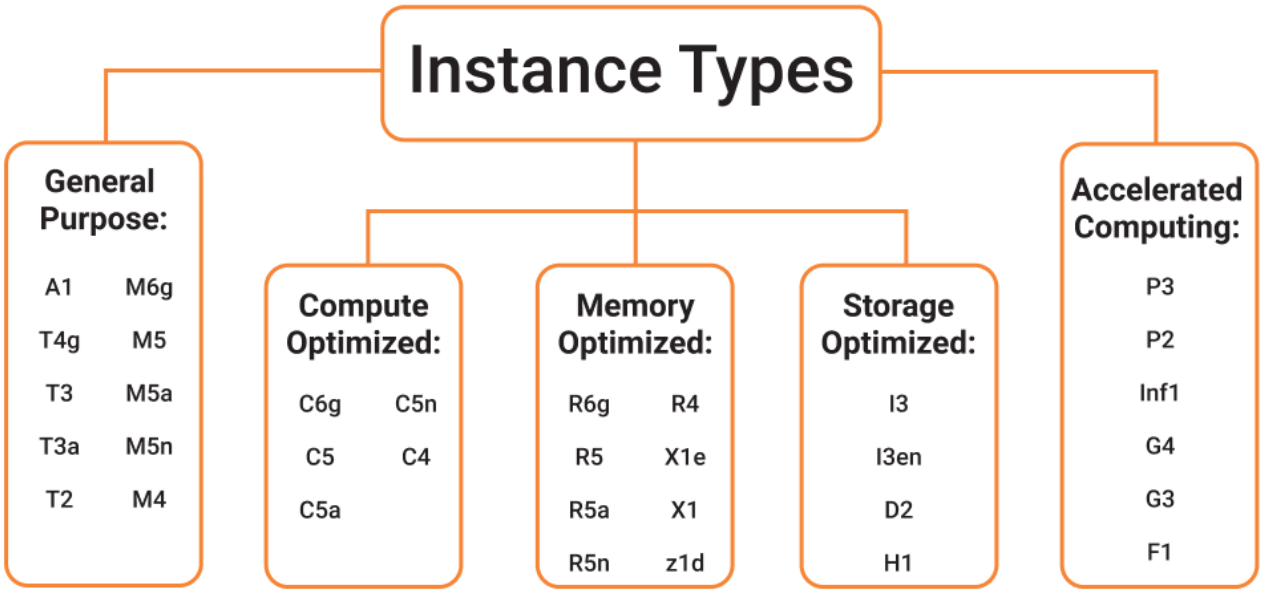




2. **Select Instance Type (Hardware)** Instance families are optimized for different workloads.

Figure 1.4 – EC2 instance type categories

	Type	Description	Mnemonic
General Purpose	a1	Good for scale-out workloads, supported by Arm	**a** is for Arm processor – or as light as **A1** steak sauce
	t-family: t3, t3a, t2	Burstable, good for changing workloads	**t** is for **tiny** or **turbo**
	m-family: m6g, m5, m5a, m5n, m4	Balanced, good for consistent workloads	**m** is for **main** or happy **medium**
Compute Optimized	c-family: c5, c5n, c4	High ratio of compute to memory	**c** is for **compute**
Memory Optimized	r-family: r5, r5a, r5n, r4	Good for in-memory databases	**r** is for **RAM**
	x1-family: x1e, x1	Good for full in-memory applications	**x** is for **xtreme**
	High memory	Good for large in-memory databases	High memory is for... high memory.
	z1d	Both high compute and high memory	**z** is for **zippy**
Accelerated Computing	p-family: p3, p2	Good for graphics processing and other GPU uses	**p** is for **pictures**
	Inf1	Support machine learning inference applications	**Inf** is for **inference**
	g-family: g4, g3	Accelerate machine learning inference and graphics-intensive workloads	**g** is for **graphics**
	f1	Customizable hardware acceleration with field programmable gate arrays (FPGAs)	**f** is for **FPGA** or **feel** as in hardware
Storage Optimized	i-family: i3, i3en	SDD-backed, balance of compute and memory	**i** is for **IOPS**
	d2	Highest disk ratio	**d** is for **dense**
	h1	HDD-backed, balance of compute and memory	**H** is for **HDD**

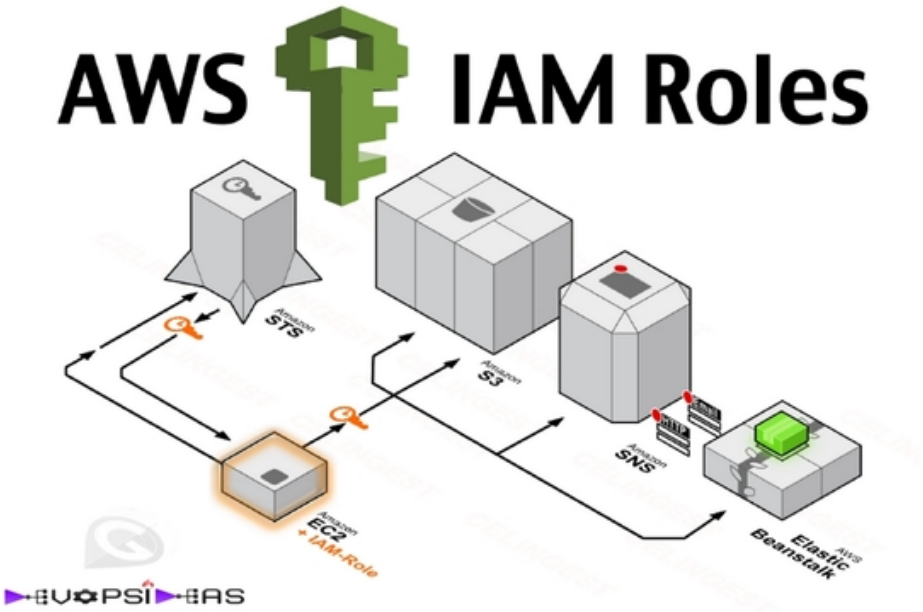


3. **Configure Networking** Use default VPC and subnet; assign public IP if internet access is required.

4. **Attach IAM Role (Optional)** Enables EC2 applications to securely access AWS services.

Figure 1.5 – IAM role attached to EC2

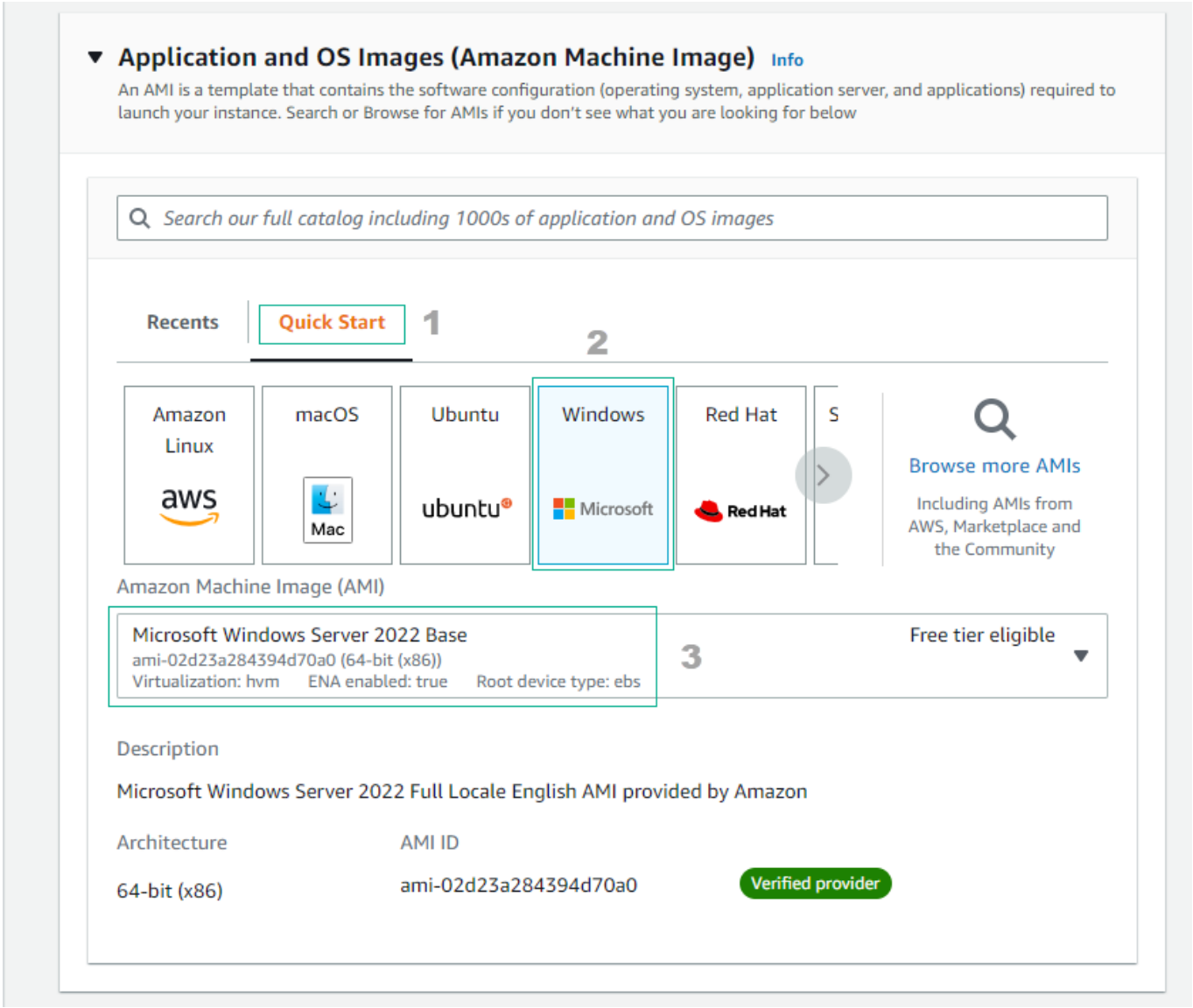
Image

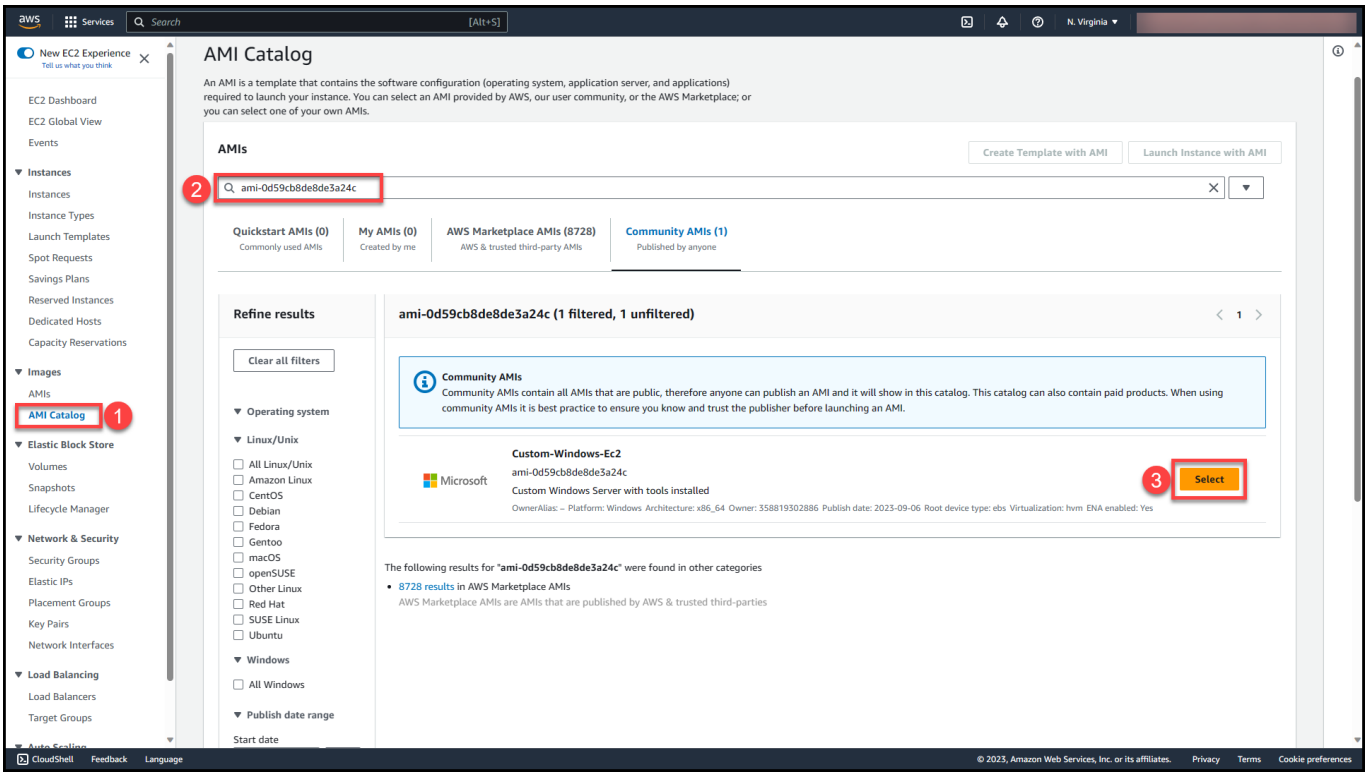


5. **User Data Script (Optional)** Executes at first boot for configuration or recovery tasks.
6. **Add Storage** Configure disk size, type, and encryption.
7. **Add Tags** Used for resource management, automation, and cost tracking.
8. **Configure Security Group (Firewall)** Define allowed inbound/outbound traffic (e.g., SSH 22, HTTP 80).
9. **Create or Select Key Pair** Required for secure access (.pem for Linux/macOS, .ppk for Windows).

Launching a Windows EC2 Instance

Figure 1.6 – Selecting Windows Server 2022 AMI

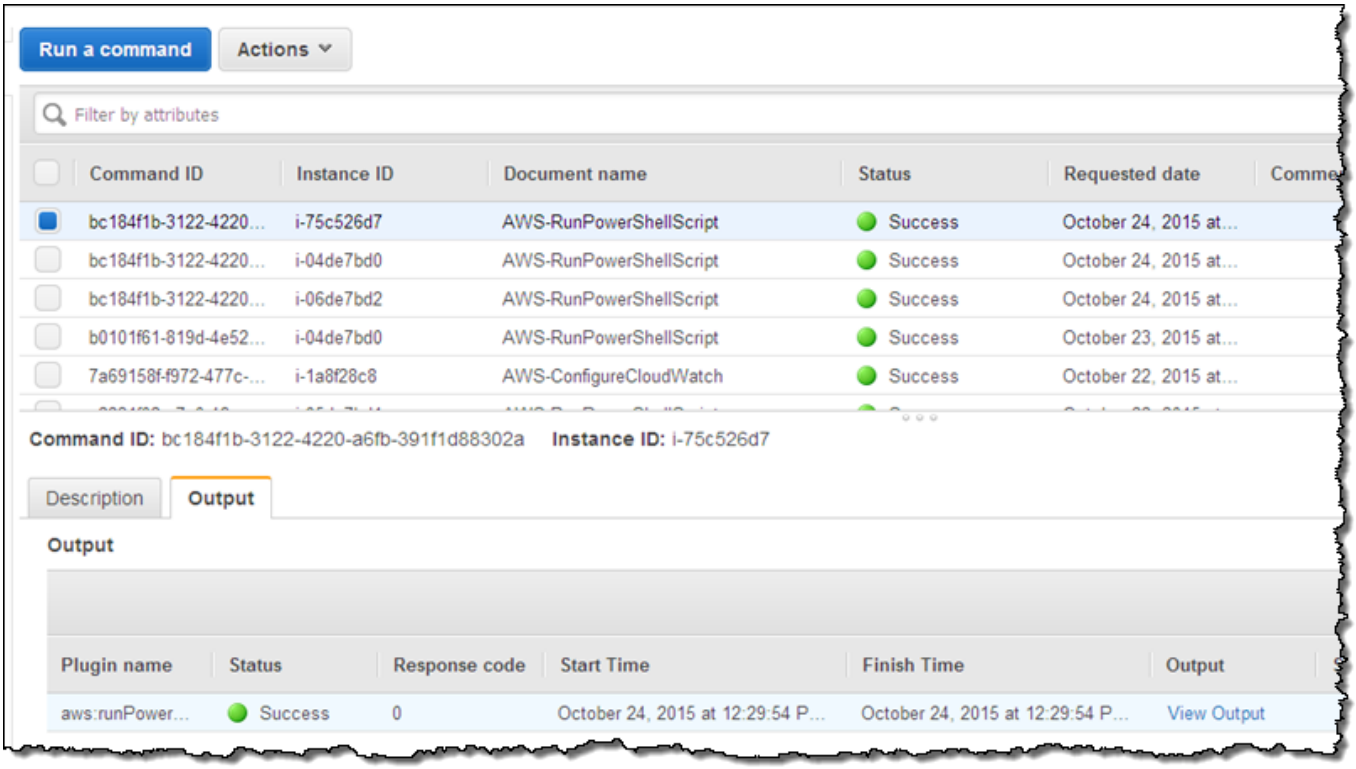

Application and OS Images (Amazon Machine Image) Info
An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below
Search our full catalog including 1000s of application and OS images
Recents
Quick Start
1
2
Amazon Linux
macOS
Ubuntu
Windows
Red Hat
S
Browse more AMIs
Including AMIs from AWS, Marketplace and the Community
Amazon Machine Image (AMI)
Microsoft Windows Server 2022 Base
ami-02d23a284394d70a0 (64-bit (x86))
Virtualization: hvm ENA enabled: true Root device type: ebs
3
Free tier eligible
Description
Microsoft Windows Server 2022 Full Locale English AMI provided by Amazon
Architecture
AMI ID
64-bit (x86)
ami-02d23a284394d70a0
Verified provider


AMI Catalog
An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. You can select an AMI provided by AWS, our user community, or the AWS Marketplace; or you can select one of your own AMIs.
AMIs
Create Template with AMI
Launch Instance with AMI
2
ami-0d59cb8de8de3a24c
Quickstart AMIs (0)
My AMIs (0)
AWS Marketplace AMIs (8728)
Community AMIs (1)
Refine results
Clear all filters
ami-0d59cb8de8de3a24c (1 filtered, 1 unfiltered)
Community AMIs
Custom-Windows-Ec2
ami-0d59cb8de8de3a24c
Custom Windows Server with tools installed
3
Select
1
AMI Catalog

Launching EC2 via CloudShell (CLI)

CloudShell is a browser-based, pre-authenticated AWS CLI environment.

Figures 1.7–1.12 – CloudShell EC2 provisioning workflow



Key steps:

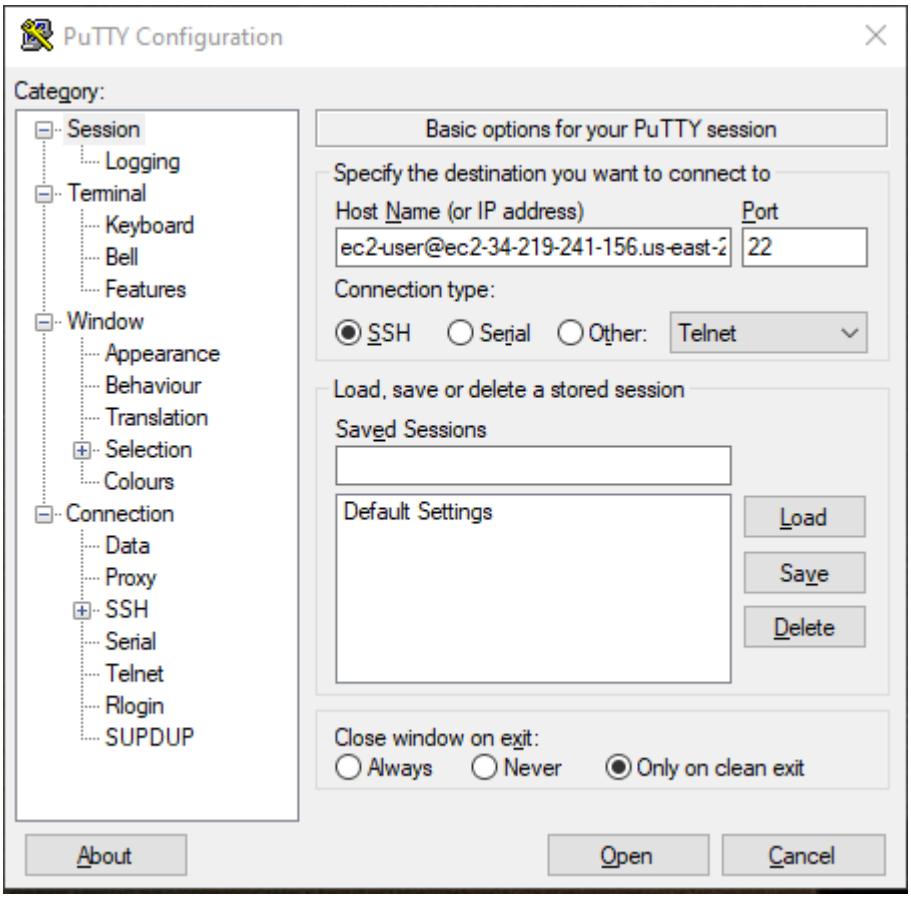
- Identify AMI ID
- Identify security group
- Identify key pair
- Run `aws ec2 run-instances`
- Verify instance details

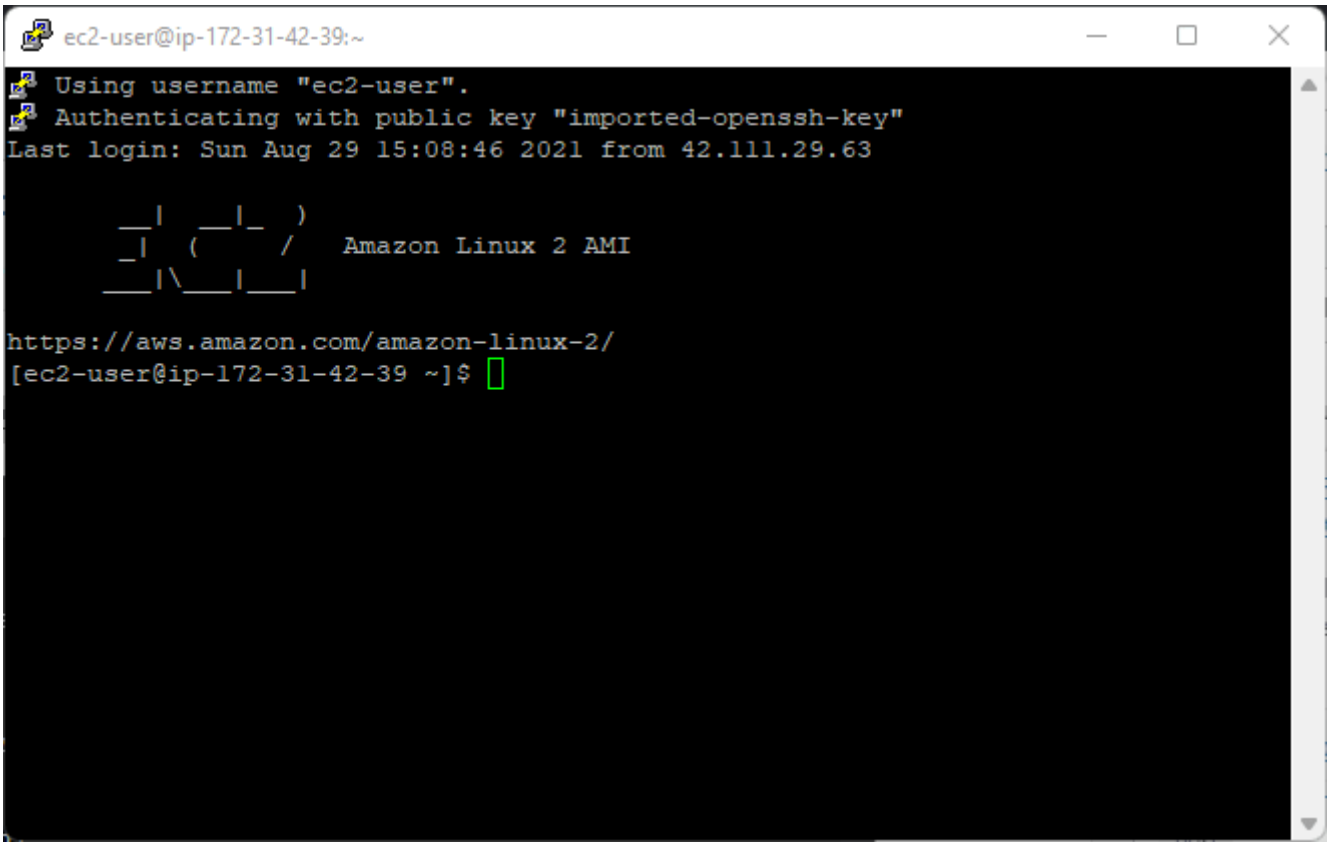
Logging into EC2 Instances

Linux EC2

- Protocol: SSH
- Tools: PuTTY (Windows), terminal (Linux/macOS)

Figures 1.13–1.15 – SSH access using PuTTY





Command example:

```
ssh -i keypair.pem ec2-user@<public-ip>
```

Windows EC2

- Protocol: RDP
- Requires decrypting administrator password using key pair

Elastic Load Balancer (ELB) and Auto Scaling Group (ASG)

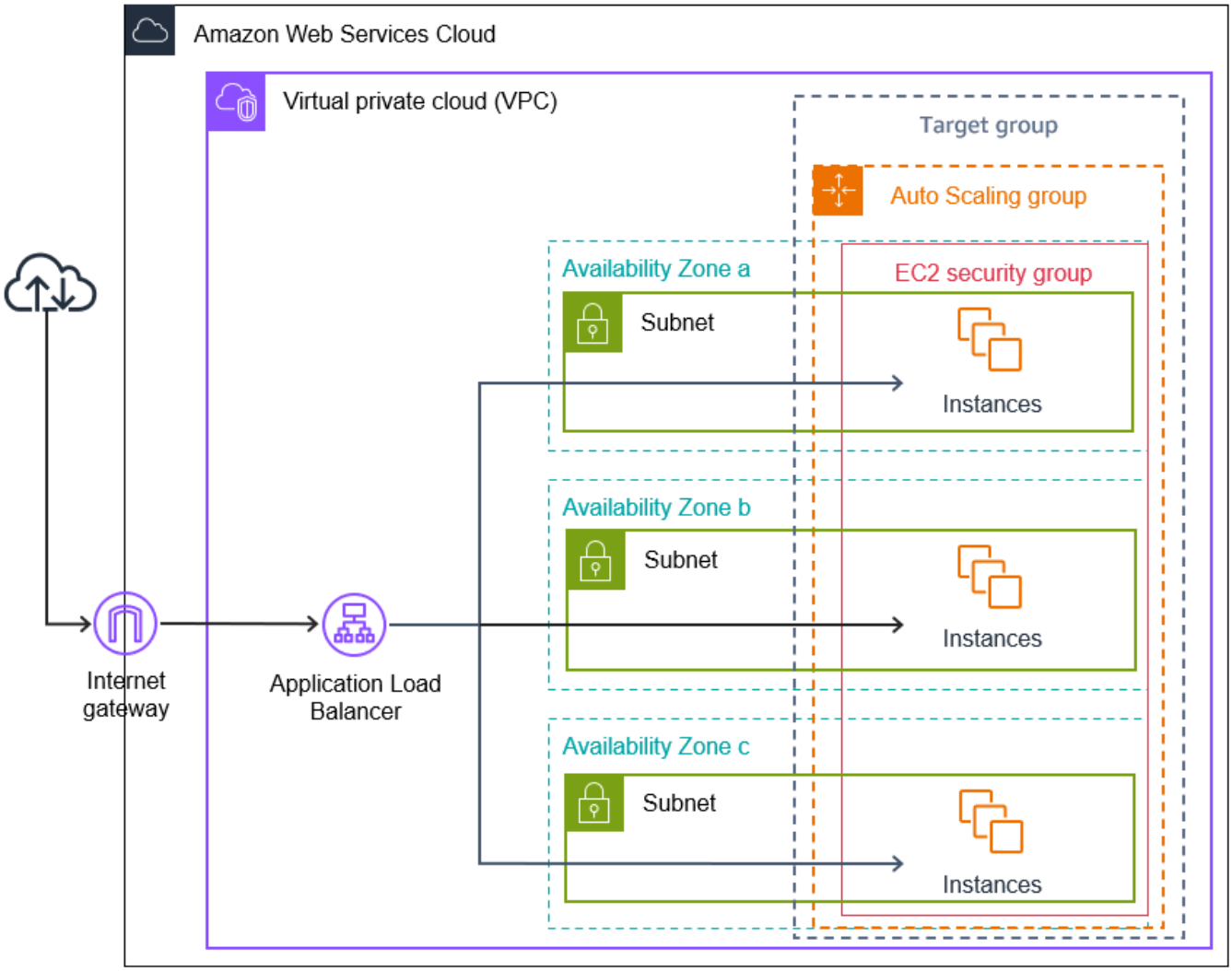
Elastic Load Balancer (ELB)

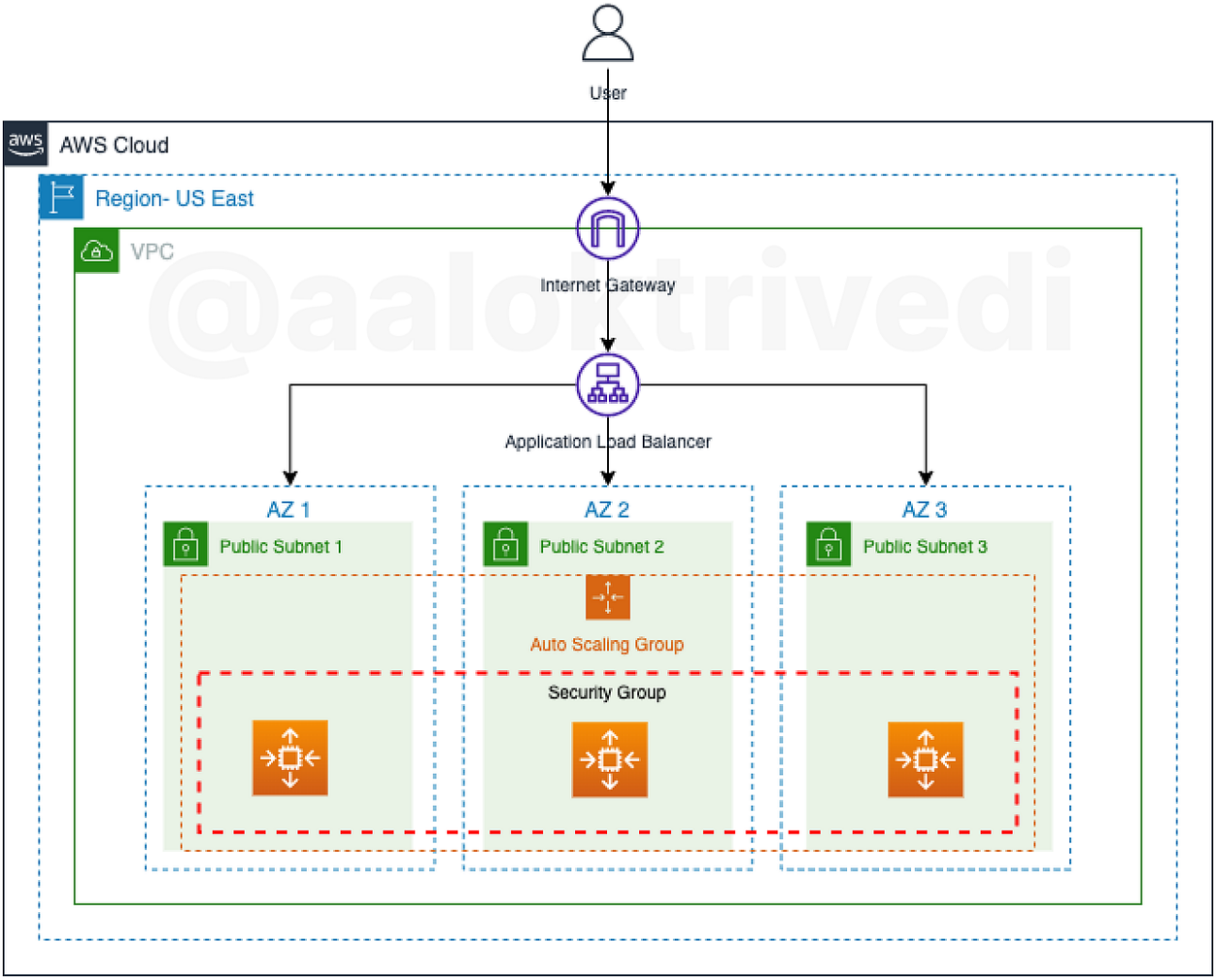
- Distributes traffic across multiple EC2 instances
- Routes requests only to healthy targets
- Improves availability and performance

Auto Scaling Group (ASG)

- Automatically adds/removes EC2 instances
- Based on metrics such as CPU utilization
- Replaces unhealthy instances
- Spans multiple AZs

Figure 1.16 – ELB + ASG architecture


Amazon Web Services Cloud
Virtual private cloud (VPC)
Target group
Auto Scaling group
EC2 security group
Availability Zone a
Subnet
Instances
Availability Zone b
Subnet
Instances
Availability Zone c
Subnet
Instances
Internet gateway
Application Load Balancer



AWS Compute Evolution: EC2 → Containers → Serverless

Containers

- Docker packages applications with dependencies
- Containers share OS kernel and start quickly
- Improve portability and deployment speed

AWS Container Services

- **Amazon ECS**: AWS-managed container orchestration
- **Amazon EKS**: Managed Kubernetes on AWS

Serverless Computing

- No server management
- Automatic scaling
- Pay per execution

AWS Lambda

- Event-driven serverless compute
- Supports multiple languages
- Tight AWS service integration
- High availability and cost efficiency

Essential AWS EC2 CloudShell / CLI Commands

This section documents the **key AWS CLI commands used when provisioning and inspecting EC2 instances via CloudShell**, along with their **purpose and parameter-level explanation**.

---

1. Describe AMIs (Amazon Machine Images)

Command

```
aws ec2 describe-images --region us-west-2
```

Purpose

- Lists all available AMIs in a specific AWS Region.
- Used to identify the **AMI ID** required to launch an EC2 instance.

Explanation

- `describe-images` queries the EC2 image catalog.
- `--region us-west-2` limits results to the **Oregon (us-west-2)** region.
- Output includes:
    - AMI ID (e.g., `ami-0ef0b498cd3fe129c`)
    - OS type
    - Architecture
    - Owner and creation date

This command is typically executed **before launching an instance** to obtain a valid AMI ID.

---

2. Describe Security Groups

Command

```
aws ec2 describe-security-groups
```

Purpose

- Lists all **Security Groups (SGs)** available in the current account and region.
- Used to identify the **security group name or ID** required during instance creation.

Explanation

- Security Groups act as **virtual firewalls**.
- Output includes:
  - Security group name (e.g., `launch-wizard-1`)
  - Inbound and outbound rules
  - VPC association

You must supply **at least one security group** when launching an EC2 instance.

---

3. Describe Key Pairs

Command

```
aws ec2 describe-key-pairs
```

Purpose

- Lists all EC2 **key pairs** available in the account.
- Used to select a key pair for **secure SSH or RDP access**.

Explanation

- Key pairs authenticate users connecting to EC2 instances.
- Output includes:
  - Key pair name (e.g., `mywestkp`)
  - Key fingerprint

If a key pair is not specified during instance creation, **remote access will not be possible**.

---

4. Launch an EC2 Instance (Core Command)

Command

```
aws ec2 run-instances \
  --image-id ami-0ef0b498cd3fe129c \
  --count 1 \
  --instance-type t2.micro \
  --key-name mywestkp \
  --security-groups launch-wizard-1 \
  --region us-west-2
```

Purpose

- **Creates (launches) a new EC2 instance** in the specified region.

Parameter Explanation

Parameter	Meaning
`run-instances`	API action to create EC2 instances
`--image-id`	Specifies the AMI (OS + base software)
`--count 1`	Number of instances to launch
`--instance-type t2.micro`	Hardware configuration (CPU, RAM)
`--key-name mywestkp`	Key pair used for secure access
`--security-groups launch-wizard-1`	Firewall rules applied to the instance
`--region us-west-2`	AWS Region where the instance is launched

Output

- Returns metadata including:
  - `InstanceId`
  - Current state (`pending`)
  - Network details

This is the **most important EC2 CLI command** and forms the foundation of infrastructure automation.

---

5. Describe EC2 Instances

Command

```
aws ec2 describe-instances
```

Purpose

- Retrieves detailed information about **all EC2 instances** in the region.

Explanation

- Used to:
  - Check instance state (running, stopped)
  - Retrieve public/private IP addresses
  - Verify instance configuration

Commonly used **after instance creation** to confirm successful provisioning.

---

6. Describe a Specific EC2 Instance

Command

```
aws ec2 describe-instances --instance-ids i-xxxxxxxxxxx
```

Purpose

- Fetches details for a **specific EC2 instance**.

Explanation

- `--instance-ids` filters output to a single instance.
- Useful when:
  - Troubleshooting
  - Automating scripts
  - Extracting IP addresses or AZ placement

---

7. SSH into a Linux EC2 Instance (Client Side)

Command

```
ssh -i keypair.pem ec2-user@<public-ip>
```

Purpose

- Establishes a **secure shell (SSH)** connection to a Linux EC2 instance.

Explanation

- `-i keypair.pem` specifies the private key file
- `ec2-user` is the default Linux username (Amazon Linux)
- `<public-ip>` is the instance's public IP address

Required Conditions

- Port **22** must be open in the Security Group
- Key file permissions must be restrictive:

```
chmod 400 keypair.pem
```

Why These Commands Matter

These CLI commands enable:

- **Repeatable infrastructure provisioning**
- **Automation and scripting**
- **Infrastructure as Code (IaC) practices**
- Full control without relying on the AWS Console UI

They are foundational for:

- DevOps pipelines
- CI/CD automation
- Production-grade AWS environments

Reference Links

- Infrastructure as Code (AWS): https://docs.aws.amazon.com/whitepapers/latest/introduction-devops-aws/infrastructure-as-code.html
- EC2 Auto Scaling with Load Balancers: https://docs.aws.amazon.com/autoscaling/ec2/userguide/autoscaling-load-balancer.html